Experiment No. - 01				
Date of Performance:				
Date of Submission:				
Implementation (04)	Understanding (04)	Punctuality & Discipline (02)	Experiment Total (10)	Sign with Date

Experiment No. - 01

1.1 Aim: To learn and practice basic and essential Unix commands

1.2 Course Outcome: Understand the working of a Unix-based operating system.

1.3 Learning Objectives: To understand the structure of the Unix file system. To learn commonly used commands

1.4 Requirements: Virtual box with Ubuntu OS

1.5 Related Theory:

Unix is a multi-user, multitasking operating system that provides a powerful command-line interface for interacting with the system. Users can execute commands through a shell, which acts as an interface between the user and the operating system.

Basic Unix commands are used for:

1. Directory Operations

1. pwd (Print Working Directory)

Syntax:

pwd

Example:

$ pwd

/home/user/Documents

2. cd (Change Directory)

Syntax:

cd directory_name

Examples:

cd Documents

cd ..

cd /home/user/Desktop

3. mkdir (Create Directory)

Syntax:

mkdir directory_name

Example:

mkdir test_folder

4. rmdir (Remove Directory)

Syntax:

rmdir directory_name

Example:

rmdir test_folder

2. File Operations

5. touch (Create File)

Syntax:

touch file_name

Example:

touch file1.txt

6. cp (Copy File)

Syntax:

cp source destination

Examples:

cp file1.txt file2.txt

cp file1.txt /home/user/Documents/

7. mv (Move/Rename File)

Syntax:

mv source destination

Examples:

mv file1.txt file2.txt (rename)

mv file1.txt Documents/ (move)

8. rm (Delete File)

Syntax:

rm file_name

Example:

rm file1.txt

3. Listing Files

9. ls (List Directory Contents)

Syntax:

ls [options]

Examples:

ls

4. Viewing File Contents

10. cat

Syntax:

cat file_name

Example:

cat file1.txt

11. more

Syntax:

more file_name

Example:

more file1.txt

(*Displays content page by page*)

12. less

Syntax:

less file_name

Example:

less file1.txt

(*Allows forward & backward navigation*)

5. File Permissions

13. chmod (Change Permissions)

Syntax:

chmod permissions file_name

Examples:

chmod 755 file1.txt

chmod +x file1.sh

6. Process Management

14. Show processes

Ps

Shows **processes running in the current terminal session**.

ps -ef

Shows **all processes on the system**.

-e → every process

-f → full formattop

15. Kill process

kill 1234

Politely asks the process with process id 1234 the process to terminate and clean up.

kill -9 1234

Forcefully terminates the process immediately.The process **cannot ignore** this signal.

7. Disk & System Info

16. Disk usage

df -h

`df` = **disk free**

Shows **filesystem-level** disk space usage.

`-h` = **human-readable** (KB, MB, GB)

du -sh mydir

`du` = **disk usage**

Shows space used by files and directories.

`-s` = **summary only** (don't list subdirectories)

`-h` = **human-readable**

These commands help users efficiently manage files and directories without using a graphical interface, making Unix highly efficient and flexible.

1.6 Conclusion:

The experiment helped in understanding and practicing basic Unix commands. It improved familiarity with the command-line interface and enhanced the ability to perform file and directory operations efficiently in a Unix environment.

Experiment No. - 02				
Date of Performance:				
Date of Submission:				
Implementation (04)	Understanding (04)	Punctuality & Discipline (02)	Experiment Total (10)	Sign with Date

Experiment No. - 02

2.1 Aim: To write and execute shell programs for performing file handling operations such as creating, reading, copying, moving, deleting files, and checking file properties in a Unix/Linux environment.

2.2 Course Outcome: Understand the basics of shell scripting in Unix/Linux. Develop shell programs to automate file handling tasks.

2.3 Learning Objectives

- To learn the structure and syntax of shell scripts.
- To perform file operations using shell commands within scripts.
- To check file existence using conditional statements.
- To automate repetitive file-related tasks efficiently.

2.4 Requirements

- Virtual box with Ubuntu OS

2.5 Related Theory

A shell provides an environment to a user to execute commands and interact with the kernel.It is the interface between application and kernel.
Types of shell are
bash-most commonly used , sh-old shell, ksh-kon shell, tsh, fish-friendly interacted shell,zsh-zee shell-extended version of shell
You can check default shell use-
$echo $0
To check shells present in system
$ cat /etc/shells

What is Shell Scripting ?

Shell script consists of a set of commands to perform a task. All the commands execute sequentially. Some task like file manipulation, program execution user interaction, automation of task etc can be done

First basic script:

```
#!/bin/bash
$echo “Hello World!”
How to go to vi editor
$vi first.sh
How to run Shell Script
$bash filename.sh
```

Example Shell Script for File Handling

```
#!/bin/bash
echo "Enter file name:"
read filename
if [ -f $filename ]
then
echo "File exists"
echo "Contents of file:"
cat $filename
else
echo "File does not exist"
fi
```

Program to Check Whether a File Exists

```
#!/bin/bash
echo "Enter file name:"
read fname
if [ -f "$fname" ]
then
echo "File exists."
else
echo "File does not exist."
fi
```

Program to Create and Write into a File

```
#!/bin/bash
echo "Enter file name:"
read fname
echo "Enter content to write:"
read content
echo "$content" > "$fname"
echo "Content written successfully."
```

Program to Count Lines, Words, and Characters

```
#!/bin/bash
echo "Enter file name:"
read fname
if [ -f "$fname" ]
then
wc "$fname"
else
echo "File not found."
fi
```

Program to Append Content to an Existing File

```
#!/bin/bash
echo "Enter file name:"
read fname
if [ -f "$fname" ]
then
echo "Enter content to append:"
read content
echo "$content" >> "$fname"
echo "Content appended successfully."
else
echo "File not found."
fi
```

Program to Display File Content Line by Line

```
#!/bin/bash
echo "Enter file name:"
read fname
if [ -f "$fname" ]
then
while read line
do
echo "$line"
done < "$fname"
else
echo "File not found."
fi
```

Program to Copy Content from One File to Another

```
#!/bin/bash
echo "Enter source file:"
read source
```

```
echo "Enter destination file:"
read dest
if [ -f "$source" ]
then
cp "$source" "$dest"
echo "File copied successfully."
else
echo "Source file not found."
fi
```

Program to Search a Word in a File

```
#!/bin/bash
echo "Enter file name:"
read fname
echo "Enter word to search:"
read word
if [ -f "$fname" ]
then
grep "$word" "$fname"
else
echo "File not found."
fi
```

Program to Delete a File

```
#!/bin/bash
echo "Enter file name to delete:"
read fname
if [ -f "$fname" ]
then
rm "$fname"
echo "File deleted successfully."
else
echo "File not found."
fi
```

2.6 Conclusion

The experiment provided practical knowledge of shell programming for file handling. It helped in understanding how to automate file operations using scripts, improving efficiency and reducing manual effort in a Unix/Linux environment.

Experiment No. - 03				
Date of Performance:				
Date of Submission:				
Implementation (04)	Understanding (04)	Punctuality & Discipline (02)	Experiment Total (10)	Sign with Date

Experiment No. - 03

3.1 Aim: To develop and execute shell scripts using grep, awk, and sed commands for efficient text processing and pattern matching in a Unix/Linux environment.

3.2 Course Outcome: Understand advanced text-processing tools in Unix/Linux. Apply grep, awk, and sed in shell scripts for data manipulation.

3.3 Learning Objectives:

- To learn pattern matching using grep.
- To understand data extraction and reporting using awk.
- To perform text editing and substitution using sed.
- To integrate these commands into shell scripts.

3.4 Requirements:

- Virtual box with Ubuntu OS

3.5 Related Theory:

Shell scripting allows automation of tasks by combining multiple commands in a script. The commands grep, awk, and sed are powerful tools used for text processing:

1. grep Command
grep stands for **Global Regular Expression Print**. It is used to **search for a specific pattern or word in a file**.
Syntax
grep pattern filename

Example
grep "error" log.txt
Explanation:
This command searches for the word **"error"** in log.txt and prints the matching lines.
Example in Shell Script
#!/bin/bash
grep "Swapna" students.txt
This script finds the name **Swapna** in the file students.txt.
Useful Options

- -i → ignore case
- -n → show line numbers
- -c → count occurrences

Example:
grep -i "hello" file.txt

2. awk Command
awk is a powerful **text processing and data extraction tool** used for **pattern scanning and reporting**.
Syntax
awk 'pattern {action}' filename
Example
awk '{print $1}' students.txt
Explanation:
Prints the **first column** of the file.
Example with Condition
awk '$3 > 50 {print $1,$3}' marks.txt
Explanation:
Prints **name and marks** where marks are greater than **50**.
Example in Shell Script
#!/bin/bash
awk '{print $1,$2}' employees.txt

3. sed Command
sed stands for **Stream Editor**. It is used to **edit text in a file automatically**, such as replacing, deleting, or inserting text.
Syntax
sed 'command' filename
Replace Example
sed 's/old/new/' file.txt
Explanation:
Replaces the first occurrence of **old** with **new** in each line.
Replace All Occurrences
sed 's/old/new/g' file.txt
Delete Line Example
sed '3d' file.txt

Explanation:
Deletes the **3rd line** of the file.
Example in Shell Script

```
#!/bin/bash
sed 's/Linux/Unix/g' file.txt
```

Combined Example in a Shell Script

```
#!/bin/bash

echo "Lines containing 'error':"
grep "error" log.txt

echo "First column of file:"
awk '{print $1}' data.txt

echo "Replacing word 'cat' with 'dog':"
sed 's/cat/dog/g' animals.txt
```

3.6 Conclusion:

The experiment helped in understanding the use of grep, awk, and sed for text processing in shell scripts.

Experiment No. - 04				
Date of Performance:				
Date of Submission:				
Implementation (04)	Understanding (04)	Punctuality & Discipline (02)	Experiment Total (10)	Sign with Date

Experiment No. - 04

4.1 Aim: To implement and analyze different CPU scheduling algorithms such as First Come First Serve (FCFS), Shortest Job First (SJF), and Priority Scheduling using C programming.

4.2 Course Outcome: Apply CPU scheduling algorithms.

4.3 Learning Objectives

- To understand process scheduling and its importance.
- To learn the working of FCFS, SJF, and Priority scheduling algorithms.
- To calculate waiting time and turnaround time.
- To compare efficiency of different scheduling techniques.
- To develop C programs for scheduling algorithms.

4.4 Requirements:

- System with C compiler.

4.5 Related Theory:

CPU scheduling is a process by which the operating system decides which process gets CPU time. The main goal is to optimize CPU utilization and minimize waiting time.

- **FCFS (First Come First Serve):**
  Processes are executed in the order of arrival. Simple but may cause long waiting time (convoy effect).

- **SJF (Shortest Job First):**
  The process with the shortest burst time is executed first. It provides minimum average waiting time but may cause starvation.
- **Priority Scheduling:**
  Each process is assigned a priority. The process with the highest priority is executed first. Lower priority processes may suffer from starvation.

4.6 Program and Output:

// C program for implementation of FCFS scheduling

```
#include <stdio.h>

int main() {
   int n;

   printf("Enter number of processes: ");
   scanf("%d", &n);

   int at[n], bt[n], wt[n], tat[n], ct[n];

   // Input arrival time and burst time
   printf("Enter Arrival Time and Burst Time:\n");
   for(int i = 0; i < n; i++) {
      printf("P%d Arrival Time: ", i + 1);
      scanf("%d", &at[i]);
      printf("P%d Burst Time: ", i + 1);
      scanf("%d", &bt[i]);
   }

   // Sorting processes based on arrival time (FCFS rule)
   for(int i = 0; i < n - 1; i++) {
      for(int j = i + 1; j < n; j++) {
         if(at[i] > at[j]) {
            // Swap arrival time
            int temp = at[i];
            at[i] = at[j];
            at[j] = temp;

            // Swap burst time
            temp = bt[i];
            bt[i] = bt[j];
            bt[j] = temp;
         }
      }
   }
```

```
    // First process completion time
    ct[0] = at[0] + bt[0];

    // Calculate completion time
    for(int i = 1; i < n; i++) {
        if(ct[i - 1] < at[i])
            ct[i] = at[i] + bt[i];  // CPU idle case
        else
            ct[i] = ct[i - 1] + bt[i];
    }

    // Calculate TAT and WT
    for(int i = 0; i < n; i++) {
        tat[i] = ct[i] - at[i];
        wt[i] = tat[i] - bt[i];
    }

    float total_wt = 0, total_tat = 0;

    printf("\nP\tAT\tBT\tCT\tTAT\tWT\n");

    for(int i = 0; i < n; i++) {
        printf("P%d\t%d\t%d\t%d\t%d\t%d\n",
                i + 1, at[i], bt[i], ct[i], tat[i], wt[i]);

        total_wt += wt[i];
        total_tat += tat[i];
    }

    printf("\nAverage Waiting Time = %.2f", total_wt / n);
    printf("\nAverage Turnaround Time = %.2f\n", total_tat / n);

    return 0;
}
```

Output

```
Enter number of processes: 3
Enter Arrival Time and Burst Time:
P1 Arrival Time: 0
P1 Burst Time: 5
P2 Arrival Time: 2
P2 Burst Time: 6
P3 Arrival Time: 3
P3 Burst Time: 10

P       AT      BT      CT      TAT     WT
P1      0       5       5       5       0
```

P2	2	6	11	9	3
P3	3	10	21	18	8

Average Waiting Time = 3.67
Average Turnaround Time = 10.67

// C program for implementation of SJF scheduling

```
#include <stdio.h>

int main() {
    int n, i, j;
    printf("Enter number of processes: ");
    scanf("%d", &n);
    int pid[n], bt[n], wt[n], tat[n];
    // Input burst times
    printf("Enter Burst Time:\n");
    for (i = 0; i < n; i++) {
        pid[i] = i + 1;
        printf("P%d: ", pid[i]);
        scanf("%d", &bt[i]);
    }
    // Sort by burst time (SJF)
    for (i = 0; i < n - 1; i++) {
        for (j = i + 1; j < n; j++) {
            if (bt[i] > bt[j]) {
                // swap burst time
                int temp = bt[i];
                bt[i] = bt[j];
                bt[j] = temp;
                // swap process id
                temp = pid[i];
                pid[i] = pid[j];
                pid[j] = temp;
            }
        }
    }
    // Calculate waiting time
    wt[0] = 0;
    for (i = 1; i < n; i++) {
        wt[i] = wt[i - 1] + bt[i - 1];
    }
    // Calculate turnaround time
    for (i = 0; i < n; i++) {
        tat[i] = wt[i] + bt[i];
    }
```

```
    // Display results
    float total_wt = 0, total_tat = 0;
    printf("\nP\tBT\tWT\tTAT\n");
    for (i = 0; i < n; i++) {
        printf("P%d\t%d\t%d\t%d\n", pid[i], bt[i], wt[i], tat[i]);
        total_wt += wt[i];
        total_tat += tat[i];
    }
    printf("\nAverage Waiting Time = %.2f", total_wt / n);
    printf("\nAverage Turnaround Time = %.2f", total_tat / n);
    return 0;
}
```

Output:

```
Enter number of processes: 3
Enter Burst Time:
P1: 5
P2: 8
P3: 2

P       BT      WT      TAT
P3      2       0       2
P1      5       2       7
P2      8       7       15

Average Waiting Time = 3.00
Average Turnaround Time = 8.00
```

// C program for implementation of Priority scheduling

```
#include <stdio.h>

int main() {
    int n, i, j;
    printf("Enter number of processes: ");
    scanf("%d", &n);
    int bt[n], pr[n], wt[n], tat[n], pid[n];
    // Input
    for(i = 0; i < n; i++) {
        pid[i] = i + 1;
        printf("\nEnter Burst Time for P%d: ", pid[i]);
        scanf("%d", &bt[i]);
        printf("Enter Priority for P%d: ", pid[i]);
        scanf("%d", &pr[i]);
    }
```

```
    // Sorting based on priority (lower value = higher priority)
    for(i = 0; i < n; i++) {
        for(j = i + 1; j < n; j++) {
            if(pr[i] > pr[j]) {
                // Swap priority
                int temp = pr[i];
                pr[i] = pr[j];
                pr[j] = temp;
                // Swap burst time
                temp = bt[i];
                bt[i] = bt[j];
                bt[j] = temp;
                // Swap process id
                temp = pid[i];
                pid[i] = pid[j];
                pid[j] = temp;
            }
        }
    }

    // Calculate Waiting Time
    wt[0] = 0;
    for(i = 1; i < n; i++) {
        wt[i] = wt[i-1] + bt[i-1];
    }
    // Calculate Turnaround Time
    for(i = 0; i < n; i++) {
        tat[i] = wt[i] + bt[i];
    }
    // Output
    printf("\nPID\tBT\tPR\tWT\tTAT\n");
    for(i = 0; i < n; i++) {
        printf("P%d\t%d\t%d\t%d\t%d\n", pid[i], bt[i], pr[i], wt[i], tat[i]);
    }
    // Average calculations
    float total_wt = 0, total_tat = 0;
    for(i = 0; i < n; i++) {
        total_wt += wt[i];
        total_tat += tat[i];
    }
    printf("\nAverage Waiting Time = %.2f", total_wt / n);
    printf("\nAverage Turnaround Time = %.2f\n", total_tat / n);

    return 0;
}
```

Output:

```
Enter number of processes: 3

Enter Burst Time for P1: 9
Enter Priority for P1: 3

Enter Burst Time for P2: 5
Enter Priority for P2: 1

Enter Burst Time for P3: 7
Enter Priority for P3: 2

PID     BT      PR      WT      TAT
P2      5       1       0       5
P3      7       2       5       12
P1      9       3       12      21

Average Waiting Time = 5.67
Average Turnaround Time = 12.67
```

4.7 Conclusion

The experiment demonstrated the implementation of FCFS, SJF, and Priority scheduling algorithms. It helped in understanding how different scheduling techniques affect process execution, waiting time, and turnaround time, highlighting their advantages and limitations.

Experiment No. - 05				
Date of Performance:				
Date of Submission:				
Implementation (04)	Understanding (04)	Punctuality & Discipline (02)	Experiment Total (10)	Sign with Date

Experiment No. - 05

5.1 Aim: To implement and analyze different page replacement algorithms such as FIFO, Optimal, and LRU using C programming.

5.2 Course Outcome (CO): Evaluate memory management schemes, including paging, segmentation, and virtual memory, using various page replacement algorithms.

5.3 Learning Objectives

- To understand virtual memory and page replacement techniques.
- To learn the working of FIFO, Optimal, and LRU algorithms.
- To calculate the number of page faults for given reference strings.
- To compare different algorithms based on performance.
- To develop C programs for memory management techniques.

5.4 Requirements

- System with C compiler.

5.5 Related Theory

In virtual memory systems, when a page is not found in memory, a **page fault** occurs. If memory is full, a page replacement algorithm decides which page to remove.

- **FIFO (First In First Out):**
  The oldest page in memory is replaced first. Simple but may suffer from Belady's anomaly.

- **Optimal Algorithm:**
  Replaces the page that will not be used for the longest time in the future. Gives minimum page faults but is not practical.
- **LRU (Least Recently Used):**
  Replaces the page that has not been used for the longest time in the past. More efficient than FIFO but requires tracking usage.

5.6 Program and Output

//C program for FIFO page replacement algorithm

```
#include <stdio.h>

int main() {
   int frames, pages, i, j, index = 0, faults = 0;

   printf("Enter number of frames: ");
   scanf("%d", &frames);

   printf("Enter number of pages: ");
   scanf("%d", &pages);

   int ref[pages], frame[frames];

   printf("Enter reference string:\n");
   for(i = 0; i < pages; i++)
      scanf("%d", &ref[i]);

   // Initialize frames
   for(i = 0; i < frames; i++)
      frame[i] = -1;

   for(i = 0; i < pages; i++) {
      int found = 0;

      for(j = 0; j < frames; j++) {
         if(frame[j] == ref[i]) {
            found = 1;
            break;
         }
      }

      if(!found) {
         frame[index] = ref[i];
         index = (index + 1) % frames;
         faults++;
      }
   }
```

```
    printf("Total Page Faults (FIFO) = %d\n", faults);

    return 0;
}
```

Output:

```
Enter number of frames: 3
Enter number of pages: 9
Enter reference string:
8 5 2 3 5 8 3 8 2
Total Page Faults (FIFO) = 5
```

//C program for Optimal Page Replacement

```
#include <stdio.h>

int main() {
    int frames, pages, i, j, k, faults = 0;

    printf("Enter number of frames: ");
    scanf("%d", &frames);

    printf("Enter number of pages: ");
    scanf("%d", &pages);

    int ref[pages], frame[frames];

    printf("Enter reference string:\n");
    for(i = 0; i < pages; i++)
        scanf("%d", &ref[i]);

    // Initialize
    for(i = 0; i < frames; i++)
        frame[i] = -1;

    for(i = 0; i < pages; i++) {
        int found = 0;

        for(j = 0; j < frames; j++) {
            if(frame[j] == ref[i]) {
                found = 1;
                break;
            }
        }
```

```
        if(!found) {
            int farthest = -1, pos = -1;

            for(j = 0; j < frames; j++) {
                int next_use = -1;

                for(k = i + 1; k < pages; k++) {
                    if(frame[j] == ref[k]) {
                        next_use = k;
                        break;
                    }
                }

                if(next_use == -1) {
                    pos = j;
                    break;
                }

                if(next_use > farthest) {
                    farthest = next_use;
                    pos = j;
                }
            }

            frame[pos] = ref[i];
            faults++;
        }
    }

    printf("Total Page Faults (Optimal) = %d\n", faults);

    return 0;
}
```

Output:

```
Enter number of frames: 3
Enter number of pages: 9
Enter reference string:
8 5 2 3 5 8 3 8 2
Total Page Faults (Optimal) = 5
```

//C program for LRU Page Replacement

```
#include <stdio.h>

int main() {
```

```
int frames, pages, i, j, pos, faults = 0;

printf("Enter number of frames: ");
scanf("%d", &frames);

printf("Enter number of pages: ");
scanf("%d", &pages);

int ref[pages], frame[frames], time[frames];

printf("Enter reference string:\n");
for(i = 0; i < pages; i++)
   scanf("%d", &ref[i]);

// Initialize
for(i = 0; i < frames; i++) {
   frame[i] = -1;
   time[i] = 0;
}

int counter = 0;

for(i = 0; i < pages; i++) {
   int found = 0;

   for(j = 0; j < frames; j++) {
      if(frame[j] == ref[i]) {
         counter++;
         time[j] = counter;
         found = 1;
         break;
      }
   }

   if(!found) {
      pos = 0;
      for(j = 1; j < frames; j++) {
         if(time[j] < time[pos])
            pos = j;
      }

      frame[pos] = ref[i];
      counter++;
      time[pos] = counter;
      faults++;
   }
```

```
    }

    printf("Total Page Faults (LRU) = %d\n", faults);

    return 0;
}
```

Output:

```
Enter number of frames: 3
Enter number of pages: 9
Enter reference string:
8 5 2 3 5 8 3 8 2
Total Page Faults (LRU) = 6
```

5.7 Conclusion:

The experiment demonstrated the implementation of FIFO, LRU, and Optimal page replacement algorithms. It helped in understanding how different strategies affect page faults. Among them, the Optimal algorithm gives the best performance, while LRU is more practical and efficient compared to FIFO.

Experiment No. - 06				
Date of Performance:				
Date of Submission:				
Implementation (04)	Understanding (04)	Punctuality & Discipline (02)	Experiment Total (10)	Sign with Date

Experiment No. - 06

6.1 Aim: To study and implement concurrent programming using processes and threads, and to understand system calls such as fork() and vfork()

6.2 Course Outcome (CO): Apply CPU scheduling algorithms and multithreading models to improve process performance and system throughput.

6.3 Learning Objectives

- To understand concurrent execution of processes and threads.
- To learn process creation using fork() and vfork().
- To create and manage threads using pthreads.
- To differentiate between processes and threads.
- To observe execution behavior in concurrent programs.

6.4 Requirements

- System with C compiler.

6.5 Related Theory

Concurrent programming allows multiple processes or threads to execute simultaneously, improving system efficiency and performance.

- **Process:**
  A process is an independent program in execution with its own memory space.

- **Thread:**
  A thread is a lightweight process that shares memory with other threads within the same process.
- **fork() system call:** Fork() is a system call which is used to create a new process. The new process created by fork() system call is called child process and the process that invoked fork() system call is called parent process. The code of the child process is the same as the code of its parent process. Once a child process is created, both parent and child processes start their execution from the next statement after fork() and both processes get executed simultaneously.
- **vfork() system call :** Vfork() is also a system call which is used to create a new process. The new process created by vfork() system call is called child process and the process that invoked vfork() system call is called parent process. The code of the child process is the same as the code of its parent process. The child process suspends execution of the parent process until the child process completes its execution as both processes share the same address space.
- **Concurrency:**
  Multiple tasks progress simultaneously.

6.6 Program and Output

// Process Creation with fork(), vfork()

```
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>

int main() {
   pid_t pid;

   // fork() example
   pid = fork();
   if (pid == 0) {
      printf("Child Process using fork (PID: %d)\n", getpid());
   } else if (pid > 0) {
      printf("Parent Process using fork (PID: %d), Child PID: %d\n", getpid(), pid);
   }

   // vfork() example
   printf("\nUsing vfork:\n");
   pid = vfork();

   if (pid == 0) {
      printf("Child Process using vfork (PID: %d)\n", getpid());
```

```
        _exit(0);  // Important for vfork
    } else {
        printf("Parent Process after vfork (PID: %d)\n", getpid());
    }
    return 0;
}
```

Output:

```
Parent Process using fork (PID: 118458), Child PID: 118459

Using vfork:
Child Process using fork (PID: 118459)

Using vfork:
Child Process using vfork (PID: 118460)
Child Process using vfork (PID: 118461)
Parent Process after vfork (PID: 118458)
Parent Process after vfork (PID: 118459)
```

//Program for pthread

```
#include <stdio.h>
#include <pthread.h>

void* thread_func(void* arg)
 {
    for(int i = 0; i < 3; i++) {
        printf("Thread running iteration %d\n", i);
    }
    return NULL;
}

int main()
{
    pthread_t t1, t2;

    pthread_create(&t1, NULL, thread_func, NULL);
    pthread_create(&t2, NULL, thread_func, NULL);

    pthread_join(t1, NULL);
    pthread_join(t2, NULL);

    printf("Main thread completed\n");

    return 0;
}
```

Output:

```
Thread running iteration 0
Thread running iteration 1
Thread running iteration 2
Thread running iteration 0
Thread running iteration 1
Thread running iteration 2
Main thread completed
```

6.7 Conclusion

The experiment demonstrated concurrent programming using processes (fork(), vfork()) and threads (pthreads). It helped in understanding process creation, thread execution, and the basic concept of concurrency.

Experiment No. - 07				
Date of Performance:				
Date of Submission:				
Implementation (04)	Understanding (04)	Punctuality & Discipline (02)	Experiment Total (10)	Sign with Date

Experiment No. - 07

7.1 Aim: To implement synchronization mechanisms using semaphores, mutex locks, and condition variables for controlling concurrent access to shared resources.

7.2 Course Outcome

Implement process synchronization techniques, and analyze deadlock conditions with appropriate prevention, avoidance, and recovery strategies.

7.3 Learning Objectives

- To understand the need for synchronization in concurrent systems.
- To implement mutex locks for mutual exclusion.
- To use semaphores for process coordination.
- To apply condition variables for thread communication.
- To prevent race conditions in shared resources.

7.4 Requirements

- System with C compiler.

7.5 Related Theory

In concurrent programming, multiple processes/threads may access shared resources simultaneously, leading to race conditions and inconsistent results. Synchronization primitives are used to control access:

- Mutex (Lock):
  Ensures only one thread enters the critical section at a time.
- Semaphore:
  A signaling mechanism used to control access to resources.
  Types:
  - Binary semaphore (0/1)
  - Counting semaphore
- Condition Variables:
  Allow threads to wait until a specific condition is met.

These mechanisms help in achieving mutual exclusion, process coordination, and deadlock control.

7.6 Program and Output

```
#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>

int shared = 0;

pthread_mutex_t mutex;
pthread_cond_t cond;
sem_t sem;

// Thread 1: Using Mutex Lock
void* mutex_example(void* arg)
{
   pthread_mutex_lock(&mutex);   // lock

   shared++;
   printf("Mutex Lock: shared = %d\n", shared);

   pthread_mutex_unlock(&mutex); // unlock
   return NULL;
}

// Thread 2: Using Semaphore
void* semaphore_example(void* arg)
{
```

```
    sem_wait(&sem);   // wait

    printf("Semaphore: Thread inside critical section\n");

    sem_post(&sem);   // signal
    return NULL;
}

// Thread 3: Using Condition Variable
void* condition_example(void* arg)
{
    pthread_mutex_lock(&mutex);

    printf("Condition Variable: Waiting...\n");
    pthread_cond_wait(&cond, &mutex);

    printf("Condition Variable: Thread resumed\n");

    pthread_mutex_unlock(&mutex);
    return NULL;
}

int main()
{
    pthread_t t1, t2, t3;

    // Initialize
    pthread_mutex_init(&mutex, NULL);
    pthread_cond_init(&cond, NULL);
    sem_init(&sem, 0, 1);

    // Create threads
    pthread_create(&t1, NULL, mutex_example, NULL);
    pthread_create(&t2, NULL, semaphore_example, NULL);
    pthread_create(&t3, NULL, condition_example, NULL);

    sleep(1); // give time for waiting thread

    // Signal condition variable
    pthread_mutex_lock(&mutex);
```

```
    pthread_cond_signal(&cond);
    pthread_mutex_unlock(&mutex);

    // Wait for threads
    pthread_join(t1, NULL);
    pthread_join(t2, NULL);
    pthread_join(t3, NULL);

    // Destroy
    pthread_mutex_destroy(&mutex);
    pthread_cond_destroy(&cond);
    sem_destroy(&sem);

    return 0;
}
```

Output:
Mutex Lock: shared = 1
Semaphore: Thread inside critical section
Condition Variable: Waiting...
Condition Variable: Thread resumed

7.7 Conclusion

The experiment demonstrated the use of synchronization primitives such as mutex locks, semaphores, and condition variables. It helped in understanding how to control concurrent access to shared resources and prevent race conditions, ensuring correct and efficient program execution.

Experiment No. - 08				
Date of Performance:				
Date of Submission:				
Implementation (04)	Understanding (04)	Punctuality & Discipline (02)	Experiment Total (10)	Sign with Date

Experiment No. - 08

8.1 Aim: To implement and understand the Producer-Consumer Problem and Banker's Algorithm using C programming.

8.2 Course Outcome (CO): Implement process synchronization techniques, and analyze deadlock conditions with appropriate prevention, avoidance, and recovery strategies.

8.3 Learning Objectives:

- To understand the concept of process synchronization.
- To implement the Producer-Consumer problem using synchronization techniques.
- To learn how deadlocks occur and how to avoid them.
- To implement the Banker's Algorithm for safe resource allocation.
- To analyze safe and unsafe system states.

8.4 Requirements

- System with C compiler.

8.5 Related Theory

Process Synchronization:

The need for synchronization originates when processes need to execute concurrently. The main purpose of synchronization is the sharing of resources without interference using mutual exclusion. The other purpose is the coordination of the process interactions in an operating system. Semaphores and monitors are the most powerful and most commonly used mechanisms to solve synchronization problems. Most operating systems provide semaphores. The

producer-consumer problem (also known as the bounded-buffer problem), and the readers-writers problem are two classical case studies considered to describe and test synchronization mechanisms.

Producer – Consumer Problem:

Producer – consumer problem, is a common paradigm for cooperating processes. A producer process produces information that is consumed by a consumer process. One solution to the producer– consumer problem uses shared memory. To allow producer and consumer processes to run concurrently, there must be available a buffer of items that can be filled by the producer and emptied by the consumer. This buffer will reside in a region of memory that is shared by the producer and consumer processes. A producer can produce one item while the consumer is consuming another item. The producer and consumer must be synchronized, so that the consumer does not try to consume an item that has not yet been produced.

Bankers algorithm

Banker's Algorithm is a resource allocation and deadlock avoidance algorithm. This algorithm tests for safety by simulating the allocation for predetermined maximum possible amounts of all resources, then makes an “s-state” check to test for possible activities, before deciding whether allocation should be allowed to continue.

In simple terms, it checks if allocation of any resource will lead to deadlock or not, OR if it is safe to allocate a resource to a process and if not then resource is not allocated to that process. Determining a safe sequence(even if there is only 1) will assure that the system will not go into a deadlock.

Banker's algorithm is generally used to find if a safe sequence exists or not. But here we will determine the total number of safe sequences and print all safe sequences.

Following are the important data structures terms applied in the banker's algorithm as follows:

Suppose n is the number of processes, and m is the number of each type of resource used in a computer system.

1. Available: It is an array of length 'm' that defines each type of resource available in the system. When Available[j] = K, means that 'K' instances of Resources type R[j] are available in the system.

2. Max: It is a [n x m] matrix that indicates each process P[i] can store the maximum number of resources R[j] (each type) in a system.

3. Allocation: It is a matrix of m x n orders that indicates the type of resources currently allocated to each process in the system. When Allocation [i, j] = K, it means that process P[i] is currently allocated K instances of Resources type R[j] in the system.

4. Need: It is an M x N matrix sequence representing the number of remaining resources for each process. When the Need[i] [j] = k, then process P[i] may require K more instances of resources typeRj to complete the assigned work.

Need[i][j] = Max[i][j] - Allocation[i][j].

5. Finish: It is the vector of the order m. It includes a Boolean value (true/false) indicating whether the process has been allocated to the requested resources, and all resources have been released after finishing its task.

The Banker's Algorithm is the combination of the safety algorithm and the resource request algorithm to control the processes and avoid deadlock in a system:

A. Safety Algorithm

It is a safety algorithm used to check whether or not a system is in a safe state or follows the safe

sequence in a banker's algorithm:

1. There are two vectors Work and Finish of length m and n in a safety algorithm.

Initialize:Work=Available

Finish[i] = false; for I = 0, 1, 2, 3, 4… n - 1.

2. Check the availability status for each type of resources [i], such as:

Need[i]<=Work

Finish[i]==false

If the i does not exist, go to step 4.

3. Work = Work +Allocation(i) // to get new resource allocation

Finish[i] = true

Go to step 2 to check the status of resource availability for the next process.

4. If Finish[i] == true; it means that the system is safe for all processes.

B. Resource Request Algorithm

A resource request algorithm checks how a system will behave when a process makes each type of resource request in a system as a request matrix. Let's create a resource request array R[i] for each process P[i]. If the Resource Requesti [j] equal to 'K', which means the process P[i] requires 'k' instances of Resources type R[j] in the system.

1. When the number of requested resources of each type is less than the Need resources, go to

step 2 and if the condition fails, which means that the process P[i] exceeds its maximum claim for the resource. As the expression suggests:

If(Request(i)<=Need)

Go to step 2;

2. And when the number of requested resources of each type is less than the available resource for each process, go to step (3). As the expression suggests:

If(Request(i)<=Available

Else Process P[i] must wait for the resource since it is not available for use.

3. When the requested resource is allocated to the process by changing state:

Available=AvailableRequest

Allocation(i)=Allocation(i)+Request(i)

Needi = Needi - Requesti

4. When the resource allocation state is safe, its resources are allocated to the process P(i). And if the new state is unsafe, the Process P (i) has to wait for each type of Request R(i) and restore the old resource-allocation state.

8.6 Program and Output

//Producer Consumer problem

```
#include <stdio.h>
int main() {
   int buffer[5], in = 0, out = 0, choice, item, count = 0;

   while(1) {
      printf("\n1. Produce  2. Consume  3. Exit");
      printf("\nEnter choice: ");
      scanf("%d", &choice);

      switch(choice) {
         case 1:
            if(count == 5) {
               printf("Buffer is Full!");
            } else {
               printf("Enter item: ");
               scanf("%d", &item);
```

```
            buffer[in] = item;
            in = (in + 1) % 5;
            count++;
            printf("Produced: %d", item);
         }
         break;

      case 2:
         if(count == 0) {
            printf("Buffer is Empty!");
         } else {
            item = buffer[out];
            out = (out + 1) % 5;
            count--;
            printf("Consumed: %d", item);
         }
         break;

      case 3:
         return 0;
      }
   }
}
```

Output:

```
1. Produce  2. Consume  3. Exit
Enter choice: 1
Enter item: 5
Produced: 5
1. Produce  2. Consume  3. Exit
Enter choice: 1
Enter item: 6
Produced: 6
1. Produce  2. Consume  3. Exit
Enter choice: 2
Consumed: 5
1. Produce  2. Consume  3. Exit
Enter choice: 3
```

```
//Banker's algorithm
#include <stdio.h>
```

```
int main() {
    int n, m, i, j;

    printf("Enter number of processes: ");
    scanf("%d", &n);

    printf("Enter number of resources: ");
    scanf("%d", &m);

    int alloc[10][10], max[10][10], need[10][10];
    int avail[10], finish[10] = {0}, safe[10];

    // Input Allocation Matrix
    printf("Enter Allocation Matrix:\n");
    for(i = 0; i < n; i++)
        for(j = 0; j < m; j++)
            scanf("%d", &alloc[i][j]);

    // Input Max Matrix
    printf("Enter Max Matrix:\n");
    for(i = 0; i < n; i++)
        for(j = 0; j < m; j++)
            scanf("%d", &max[i][j]);

    // Input Available Resources
    printf("Enter Available Resources:\n");
    for(i = 0; i < m; i++)
        scanf("%d", &avail[i]);

    // Calculate Need Matrix
    for(i = 0; i < n; i++)
        for(j = 0; j < m; j++)
            need[i][j] = max[i][j] - alloc[i][j];

    int count = 0;

    // Find Safe Sequence
    while(count < n) {
        int found = 0;
```

```
        for(i = 0; i < n; i++) {
            if(finish[i] == 0) {
                int safe_flag = 1;

                for(j = 0; j < m; j++) {
                    if(need[i][j] > avail[j]) {
                        safe_flag = 0;
                        break;
                    }
                }

                if(safe_flag) {
                    for(j = 0; j < m; j++)
                        avail[j] += alloc[i][j];

                    safe[count++] = i;
                    finish[i] = 1;
                    found = 1;
                }
            }
        }

        if(found == 0)
            break;
    }

    // Output Result
    if(count == n) {
        printf("Safe Sequence: ");
        for(i = 0; i < n; i++)
            printf("P%d ", safe[i]);
    } else {
        printf("System is NOT in safe state");
    }

    return 0;
}
```

Output:

Enter number of processes: 5

Enter number of resources: 3
Enter Allocation Matrix:
1 1 2
2 1 2
4 0 1
0 2 0
1 1 2
Enter Max Matrix:
4 3 3
3 2 2
9 0 2
7 5 3
1 1 2
Enter Available Resources:
2 1 0
Safe Sequence: P1 P4 P0 P2 P3

8.7 Conclusion

The Producer-Consumer problem demonstrates the importance of synchronization in shared resources.The Banker's Algorithm helps in avoiding deadlocks by ensuring safe resource allocation.

Experiment No. - 09				
Date of Performance:				
Date of Submission:				
Implementation (04)	Understanding (04)	Punctuality & Discipline (02)	Experiment Total (10)	Sign with Date

Experiment No. - 09

9.1 Aim: To implement and compare different memory allocation algorithms: First Fit, Best Fit, and Worst Fit using C programming.

9.2 Course Outcome (CO): Evaluate memory management schemes, including allocation techniques for efficient utilization of memory.

9.3 Learning Objectives:

- To understand dynamic memory allocation in operating systems.
- To learn different memory allocation strategies.
- To implement First Fit, Best Fit, and Worst Fit algorithms.
- To compare memory utilization efficiency of each method.
- To analyze fragmentation in memory allocation.

9.4 Requirements

- System with C compiler.

9.5 Related Theory

The main memory is partitioned into a set of non-overlapping blocks is called as fixed partitioning. Partitions can be of equal run equalization. A process can be loaded into a partition with high or equal size. In Dynamic Partitioning, Partitions are of variable length and number. Each process is allocated a contiguous memory block of exactly the size it requires -no internal

fragmentation. Termination of a process releases its memory block. Fixed Partitions suffer from in efficient memory use-any process, no matter how small, occupies an entire partition.This was called Internal Fragmentation. Eventually, main memory form shots too small to hold any process. This is called External fragmentation. Total memory space may exist to satisfy a request but it is not contiguous. Compaction reduces external fragmentation by shuffling
memory contents to place all free memory together in one large block.

1) First Fit

In the first fit approach is to allocate the first free partition or hole large enough which can accommodate the process. It finishes after finding the first suitable free partition.

Advantage: Fastest algorithm because it searches as little as possible.

Disadvantage: The remaining unused memory is left after all ovations become wasted if it is too small. Thus requests for larger memory requirements cannot be accomplished.

2) Best Fit

The best fit deals with allocating the smallest free partition which meets the requirement of the requesting process. This algorithm first searches the entire list of free partitions and considers the smallest hole that is adequate. It then tries to find a hole which is close to the actual process size needed.

Advantage: Memory utilization is much better than first fit as it searches the smallest free partition first available.

Disadvantage: It is slower and may even end to fill up memory within useless holes.

3) Worst fit

In worst fit approach is to locate largest available free portion so that the portion left will be big enough to be useful. It is the reverse of best fit.

Advantage: Reduces the rate of production of small gaps.

Disadvantage: If a process requiring large memory arrives at a later stage then it cannot be accommodated when the largest hole is already split and occupied.

9.7 Program and Output

```
#include <stdio.h>
int main() {
   int blocks[10], processes[10];
   int i, j, nb, np;
```

```
printf("Enter number of memory blocks: ");
scanf("%d", &nb);

printf("Enter number of processes: ");
scanf("%d", &np);

printf("Enter size of blocks:\n");
for(i = 0; i < nb; i++)
   scanf("%d", &blocks[i]);

printf("Enter size of processes:\n");
for(i = 0; i < np; i++)
   scanf("%d", &processes[i]);

// First Fit
printf("\nFirst Fit:\n");
int temp[10];
for(i = 0; i < nb; i++)
   temp[i] = blocks[i];

for(i = 0; i < np; i++) {
   for(j = 0; j < nb; j++) {
      if(temp[j] >= processes[i]) {
         printf("Process %d allocated to Block %d\n", i+1, j+1);
         temp[j] -= processes[i];
         break;
      }
   }
}

// Best Fit
printf("\nBest Fit:\n");
for(i = 0; i < nb; i++)
   temp[i] = blocks[i];

for(i = 0; i < np; i++) {
   int best = -1;
   for(j = 0; j < nb; j++) {
      if(temp[j] >= processes[i]) {
```

```
            if(best == -1 || temp[j] < temp[best])
                best = j;
        }
    }
    if(best != -1) {
        printf("Process %d allocated to Block %d\n", i+1, best+1);
        temp[best] -= processes[i];
    }
}

// Worst Fit
printf("\nWorst Fit:\n");
for(i = 0; i < nb; i++)
    temp[i] = blocks[i];

for(i = 0; i < np; i++) {
    int worst = -1;
    for(j = 0; j < nb; j++) {
        if(temp[j] >= processes[i]) {
            if(worst == -1 || temp[j] > temp[worst])
                worst = j;
        }
    }
    if(worst != -1) {
        printf("Process %d allocated to Block %d\n", i+1, worst+1);
        temp[worst] -= processes[i];
    }
}

return 0;
}
```

Output:

```
Enter number of memory blocks: 3
Enter number of processes: 3

Blocks: 100 500 200
Processes: 120 50 180

First Fit:
Process 1 allocated to Block 2
```

Process 2 allocated to Block 1
Process 3 allocated to Block 2

Best Fit:
Process 1 allocated to Block 3
Process 2 allocated to Block 1
Process 3 allocated to Block 2

Worst Fit:
Process 1 allocated to Block 2
Process 2 allocated to Block 2
Process 3 allocated to Block 2

9.8 Conclusion

The experiment demonstrates how **First Fit, Best Fit, and Worst Fit** allocate memory to processes. Each method has different efficiency and fragmentation behavior, showing the importance of choosing the right allocation technique for better memory utilization.

Experiment No. - 10				
Date of Performance:				
Date of Submission:				
Implementation (04)	Understanding (04)	Punctuality & Discipline (02)	Experiment Total (10)	Sign with Date

Experiment No. - 10

10.1 Aim: To implement and analyze different **disk scheduling algorithms**: FCFS, SSTF, SCAN, and C-SCAN .

10.2 Course Outcome (CO): Analyze file system organization, storage allocation methods, and disk scheduling techniques for efficiency and performance.

10.3 Learning Objectives

- To understand disk scheduling in operating systems.
- To implement **FCFS, SSTF, SCAN, and C-SCAN algorithms**.
- To calculate total seek time for each algorithm.

10.4 Requirements

- System with C compiler.

10.5 Related Theory

Disk Scheduling

Disk scheduling algorithms determine the order in which disk I/O requests are serviced to reduce **seek time** and improve performance.

1. FCFS (First Come First Serve)

- Processes requests in the order they arrive.

- Simple but may result in high seek time.

2. SSTF (Shortest Seek Time First)

- Selects the request closest to the current head position.
- Reduces seek time but may cause starvation.

3. SCAN (Elevator Algorithm)

- The disk arm moves in one direction servicing requests, then reverses direction.
- Provides better performance than FCFS.

4. C-SCAN (Circular SCAN)

- Head moves in one direction only.
- After reaching the end, it jumps to the beginning.
- Provides uniform wait time.

10.6 Program and Output

1. FCFS (First Come First Serve)

```
#include <stdio.h>
#include <stdlib.h>

int main() {
  int req[20], n, head, i, seek = 0;

  printf("Enter number of requests: ");
  scanf("%d", &n);

  printf("Enter request sequence:\n");
  for(i = 0; i < n; i++)
    scanf("%d", &req[i]);

  printf("Enter initial head position: ");
  scanf("%d", &head);

  for(i = 0; i < n; i++) {
    seek += abs(req[i] - head);
    head = req[i];
```

```
  }

  printf("Total Seek Time (FCFS): %d", seek);

  return 0;
}
```

Output:

```
Enter number of requests: 7
Enter request sequence:
82 170 43 140 24 16 190
Enter initial head position: 50
Total Seek Time (FCFS): 642
```

2. SSTF (Shortest Seek Time First)

```
#include <stdio.h>
#include <stdlib.h>

int main() {
  int req[20], visited[20] = {0};
  int n, head, i, j, seek = 0;

  printf("Enter number of requests: ");
  scanf("%d", &n);

  printf("Enter request sequence:\n");
  for(i = 0; i < n; i++)
    scanf("%d", &req[i]);

  printf("Enter initial head position: ");
  scanf("%d", &head);

  for(i = 0; i < n; i++) {
    int min = 9999, index = -1;

    for(j = 0; j < n; j++) {
      if(!visited[j]) {
        int dist = abs(req[j] - head);
        if(dist < min) {
          min = dist;
          index = j;
```

```
            }
        }
    }

    visited[index] = 1;
    seek += min;
    head = req[index];
  }

  printf("Total Seek Time (SSTF): %d", seek);

  return 0;
}
```

Output:

```
Enter number of requests: 7
Enter request sequence:
82 170 43 140 24 16 190
Enter initial head position: 50
Total Seek Time (SSTF): 208
```

3. SCAN (Elevator Algorithm)

```
#include <stdio.h>
#include <stdlib.h>

int main() {
  int req[20], n, head, size = 200;
  int i, j, seek = 0;

  printf("Enter number of requests: ");
  scanf("%d", &n);

  printf("Enter request sequence:\n");
  for(i = 0; i < n; i++)
    scanf("%d", &req[i]);

  printf("Enter initial head position: ");
  scanf("%d", &head);

  // Sort requests
  for(i = 0; i < n - 1; i++)
```

```
    for(j = 0; j < n - i - 1; j++)
        if(req[j] > req[j + 1]) {
            int temp = req[j];
            req[j] = req[j + 1];
            req[j + 1] = temp;
        }

  int pos;
  for(i = 0; i < n; i++)
    if(req[i] > head) {
        pos = i;
        break;
    }

  // Move right
  for(i = pos; i < n; i++) {
    seek += abs(req[i] - head);
    head = req[i];
  }

  // Go to end
  seek += abs((size - 1) - head);
  head = size - 1;

  // Move left
  for(i = pos - 1; i >= 0; i--) {
    seek += abs(req[i] - head);
    head = req[i];
  }

  printf("Total Seek Time (SCAN): %d", seek);

  return 0;
}
```

Output:

```
Enter number of requests: 7
Enter request sequence:
82 170 43 140 24 16 190
Enter initial head position: 50
Total Seek Time (SCAN): 332
```

4. C-SCAN (Circular SCAN)

```
#include <stdio.h>
#include <stdlib.h>

int main() {
  int req[20], n, head, size = 200;
  int i, j, seek = 0;

  printf("Enter number of requests: ");
  scanf("%d", &n);

  printf("Enter request sequence:\n");
  for(i = 0; i < n; i++)
     scanf("%d", &req[i]);

  printf("Enter initial head position: ");
  scanf("%d", &head);

  // Sort requests
  for(i = 0; i < n - 1; i++)
     for(j = 0; j < n - i - 1; j++)
        if(req[j] > req[j + 1]) {
           int temp = req[j];
           req[j] = req[j + 1];
           req[j + 1] = temp;
        }

  int pos;
  for(i = 0; i < n; i++)
     if(req[i] > head) {
        pos = i;
        break;
     }

  // Move right
  for(i = pos; i < n; i++) {
     seek += abs(req[i] - head);
     head = req[i];
  }
```

```
    // Jump to start
    seek += abs((size - 1) - head);
    seek += (size - 1);
    head = 0;

    // Move right again
    for(i = 0; i < pos; i++) {
        seek += abs(req[i] - head);
        head = req[i];
    }

    printf("Total Seek Time (C-SCAN): %d", seek);

    return 0;
}
```

Output:

```
Enter number of requests: 8
Enter request sequence:
98 183 41 122 14 124  65 67
Enter initial head position: 53
Total Seek Time (C-SCAN): 386
```

10.7 Conclusion

Disk scheduling algorithms significantly affect system performance. SSTF and SCAN provide better efficiency than FCFS, while C-SCAN ensures uniform waiting time. Choosing the right algorithm helps in minimizing seek time and improving disk performance.